期末总复习

Week 1 —— transport layer — OSI + TCP +UDP(1页)

week 2 —— network layer - IP protocol + 路由算法 (link state (OSPF)
Distance vector (RIP)) + Subnet + ARP/RARP

week 3 —— data link layer

week 4 —— hardware layer — entropy、G/H
Haffiman code

4 layer关系是：

transport layer 将应用层传下来的data加上header封装成报文段

发 通过process与process间通信传到network层对应process，

network层给data加上IP header 再封装，给data link layer

data link层给IP data加首尾封装成帧(有双方MAC)，给hardware

hardware层将帧转化成电、光信号通过物理介质传输

收 hardware层收物理信号转换成data link layer的帧

data link层收到后查完整+正确，去首尾提取IP datagram

network层去除IP header提取报文段，判断是否给自己的，是就给Transport

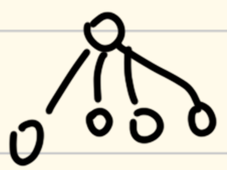
transport层根据header信息(port number)给相应进程

week 1

1. networks
   - local area
     - LAN：连有限区域内的设备和其他terminals（教室、办公室）
       privately owned，允许 resource share
   - metropolitan area
     - MAN：用于连 numbers of LANs spread around（城市范围）
   - wide area
     - WAN：提供 data transmission over large geographical areas（国家/洲）
       基于 circuit switching、packet switching 实现
   - personal area
     - PAN：data transmission over short range、
       low power、low cost small networks

transmission mode（定义2个相连设备间信号流方向）
- simplex：单向
- half-Duplex：each station 都可发可收，但同时仅1个能发
- full-Duplex：both stations 可同时发、收

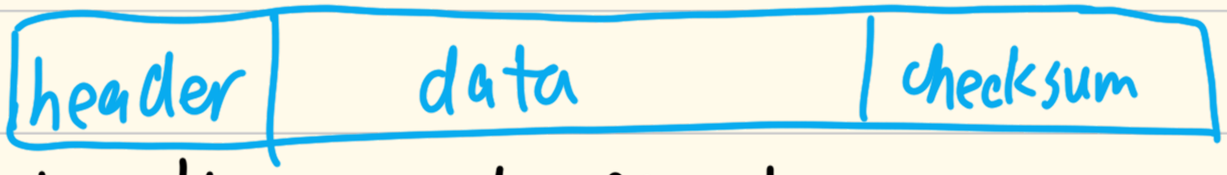
line configuration（多个device连到line的方式）
- point to point：2个device间 dedicated line
- point to multipoint 2个以上device share a single link

topology（the way a network is laid out）
- peer-to-peer
- Mesh topology：每2个device间都有专有link
- star topology：每个device与一个专用的point点对点连 
- tree topology：大多数device连secondary hub，所有secondary hub连到central hub
- Bus topology：node通过drop lines和taps连到bus cable
- ring topology：每个device有1条专用的点对点link
- hybrid topology：混和拓扑

circuit switching：terminal device间通信必须先建立通道，再通信

packet switching：按顺序发data，一次发1个packet

PDU: protowl control information



ARQ: automatic repeat request

① stop-and-wait (idle-request)

transmiter 发个data, 等receiver 发回ACK 再发下一个……

若ACK丢失, 则等time-out 令transmitter再发Data

data、ACK要有SN

② continuous request

transmiter 一直发data, 不等receiver 发ACK

有window 控制transmiter 发data 速度, 发回哪个ACK, window中对应

Data drop掉, window 前移1位

③ go-back-N (错误控制策略)

transmiter一直发data不等ACK，但当前面第n位ACK未收到(time-out)

transmiter回退到第n位重传n后面的所有data

此时重传的data也占着window size

④ selective re-transmission (错误控制策略)

receiver不会发很多ACK，receiver发ACK=n时表示n位之前的data全收到

transmiter的window可drop n位前所有data，前进n位

且receiver发的ACK是连续收到的data的最

Flow control { stop / go
slide window

protowl: sender、receiver及中间设备都要遵循的有效通信规则

standard: { open systems interconnection

程序protowl，是用于开发协议标准的框架——用分层

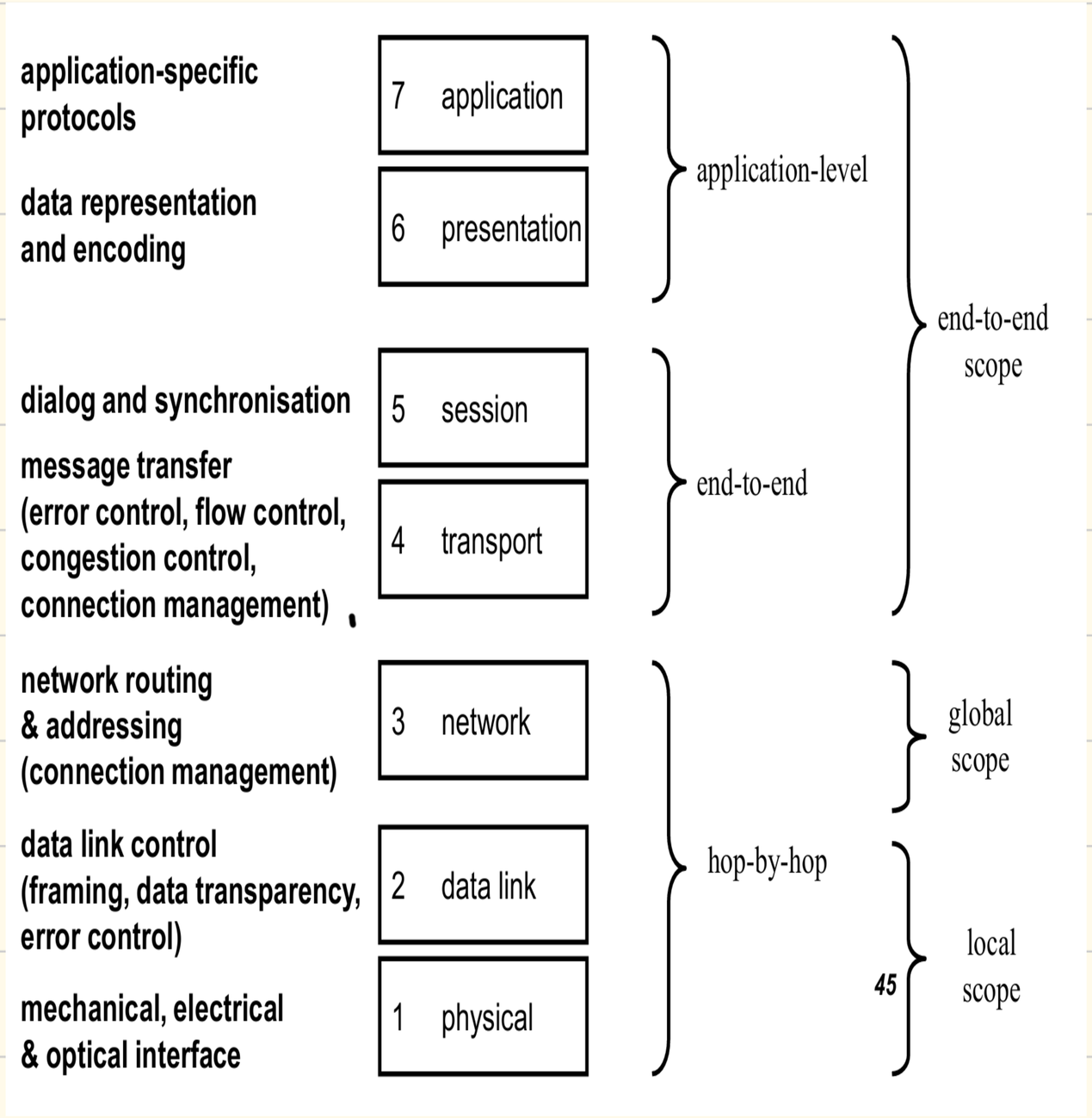
每层用下层的service 并向上层提供service

protowl layering原则：每层去能执行2个相关任务

sender、receiver 每层的2个object 相同

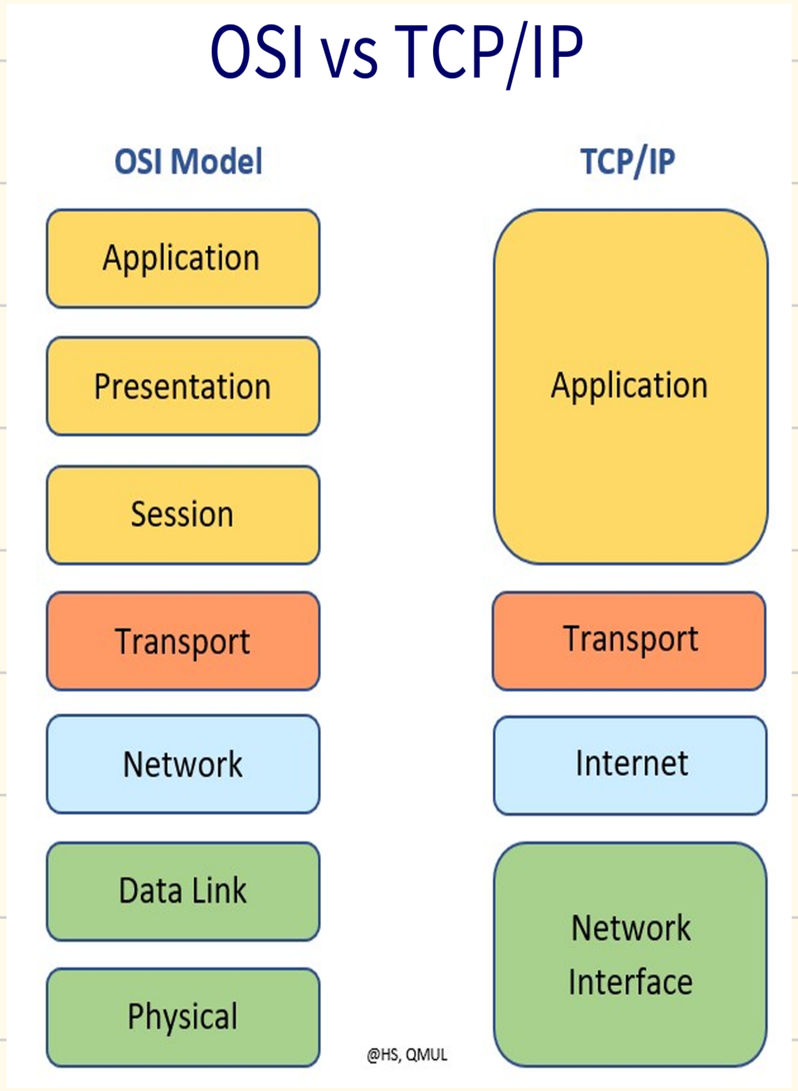
看week 1 ppt 46~48页

Layer	Function	Example
Application (7)	Services that are used with end user applications	SMTP,
Presentation (6)	Formats the data so that it can be viewed by the user Encrypt and decrypt	JPG, GIF, HTTPS, SSL, TLS
Session (5)	Establishes/ends connections between two hosts	NetBIOS, PPTP
Transport (4)	Responsible for the transport protocol and error handling	TCP, UDP
Network (3)	Reads the IP address form the packet.	Routers, Layer 3 Switches
Data Link (2)	Reads the MAC address from the data packet	Switches
Physical (1)	Send data on to the physical wire.	Hubs, NICS, Cable



TCP/IP architecture: TCP定义程序如何在网络上创建通信通道，管理如何传输前将data分割成小packet，并在destination重新组装

IP：定义如何寻址、路由每个packet



自己看 week1 52~54页

transport layer

TCP: transmission control protocol

slow + highest

① connection-oriented communication

data transmission 前要先 establish connection

each delivered message 要发 ACK

② reliable connection

确认 发、收 data 相同

丢失/损坏 data 要 retransmit

③ flow control

通过 sliding window mechanism

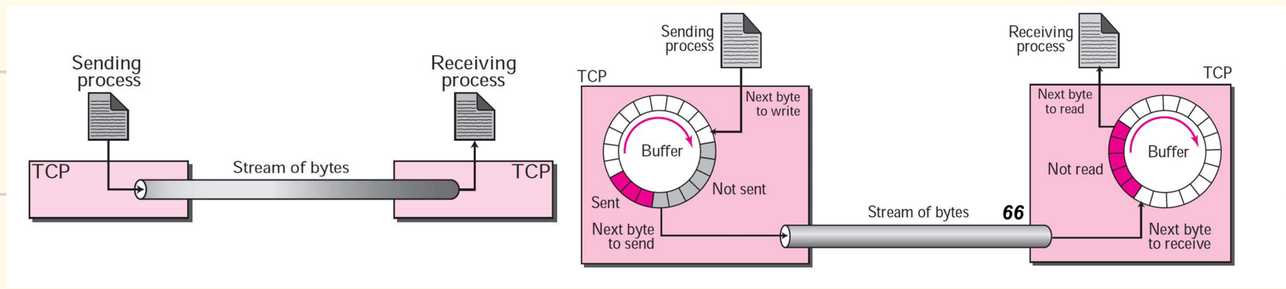
④ congestion control

⑤ error detection 通过 header 的 checksum

⑥ Byte-stream service full-duplex buffered communication

允许 process 以 byte stream 形式发/收 data

用 buffer 管理双方速度差异

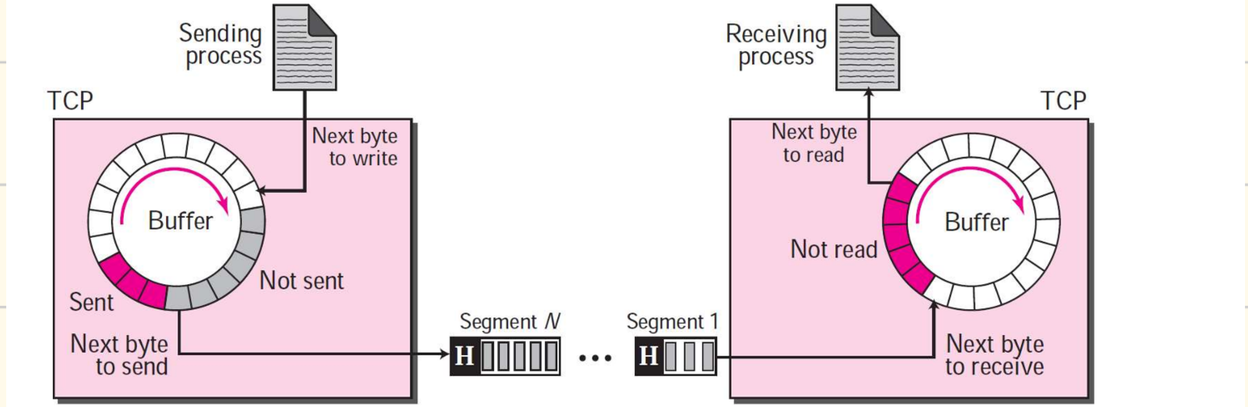


用sN 按顺序发segment

① TCP on source host buffer enough bytes 来 fill a packet 以 segment

② TCP 给每个 segment 加 header 交给 its peer on destination host

③ destination host 将 packet 中的 content 放入 receiver buffer, 空闲时 read it



URG: urgent data

ACK: 表示 Acknowledgment Number 有效

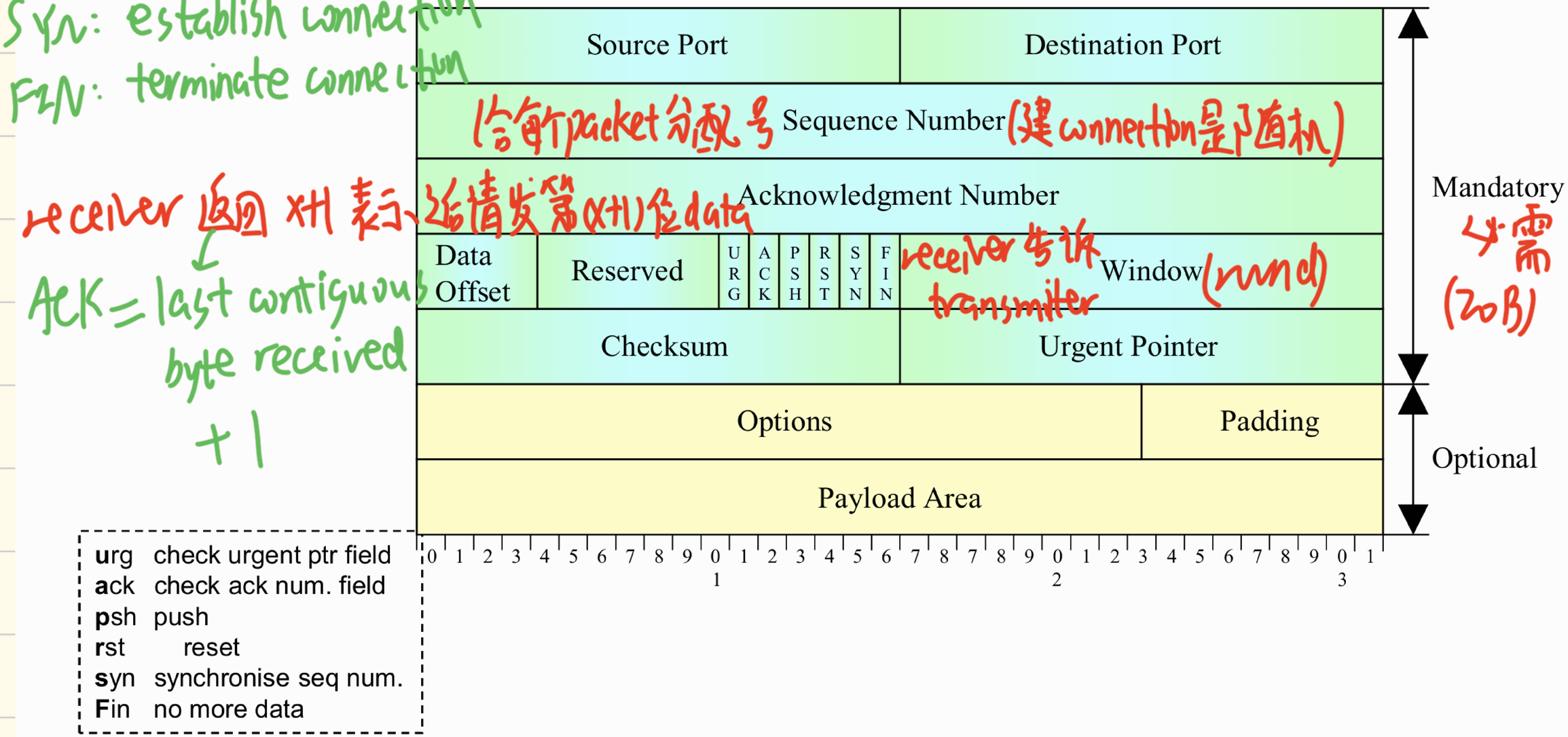
PSH: transmitter 通知 receiver 该 segment 之前的 data 尽快交给 application 而不要等缓冲区满

RST: receiver 收到不希望收的 segment，想中止 connection

SYN: establish connection

FIN: terminate connection

TCP Segment Format



给每个 packet 分配号 Sequence Number (建 connection 是随机)

receiver 返回 对 表示 近情发第(x+1)位 data

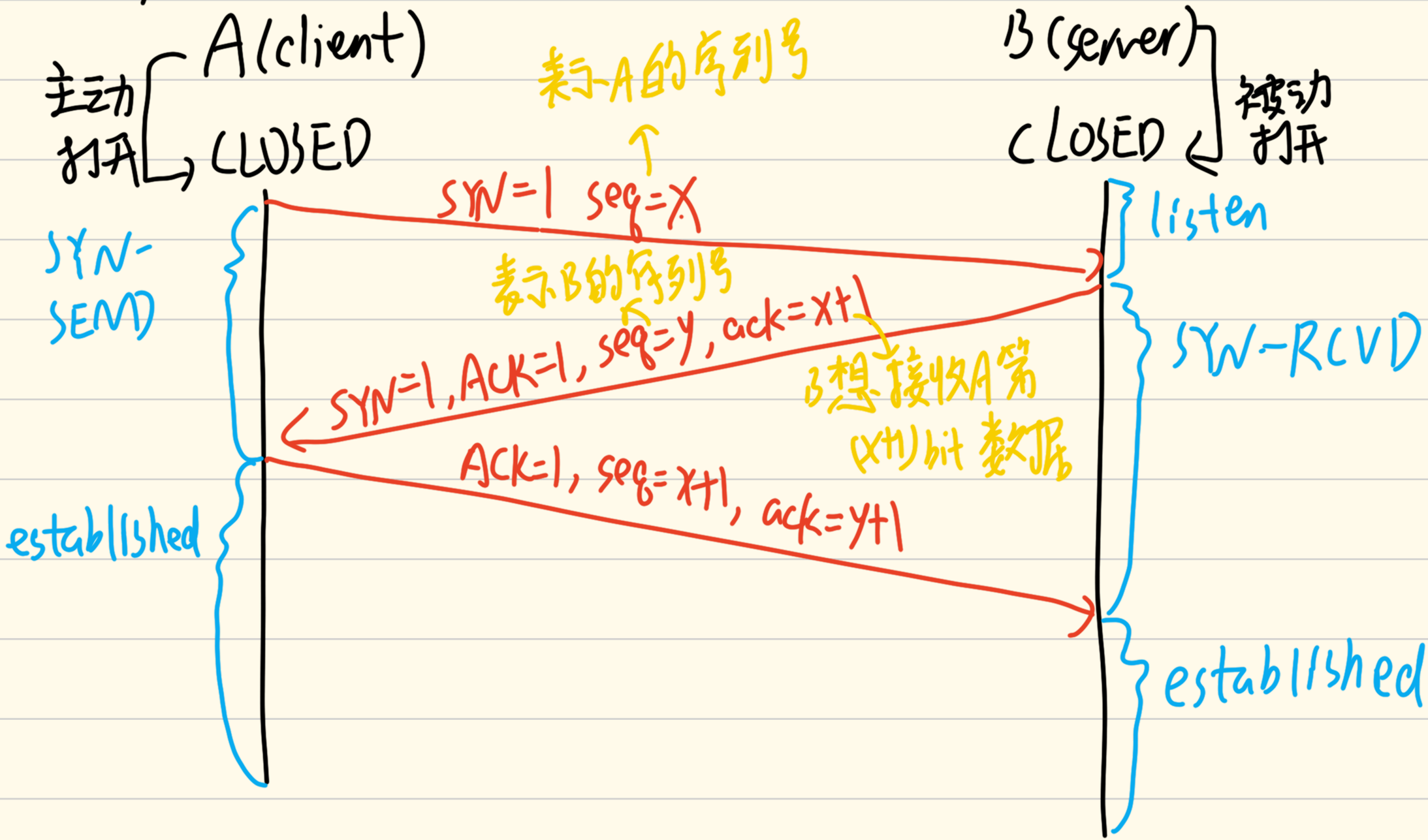
ACK = last contiguous byte received + 1

receiver 告诉 transmiter (rwnd)

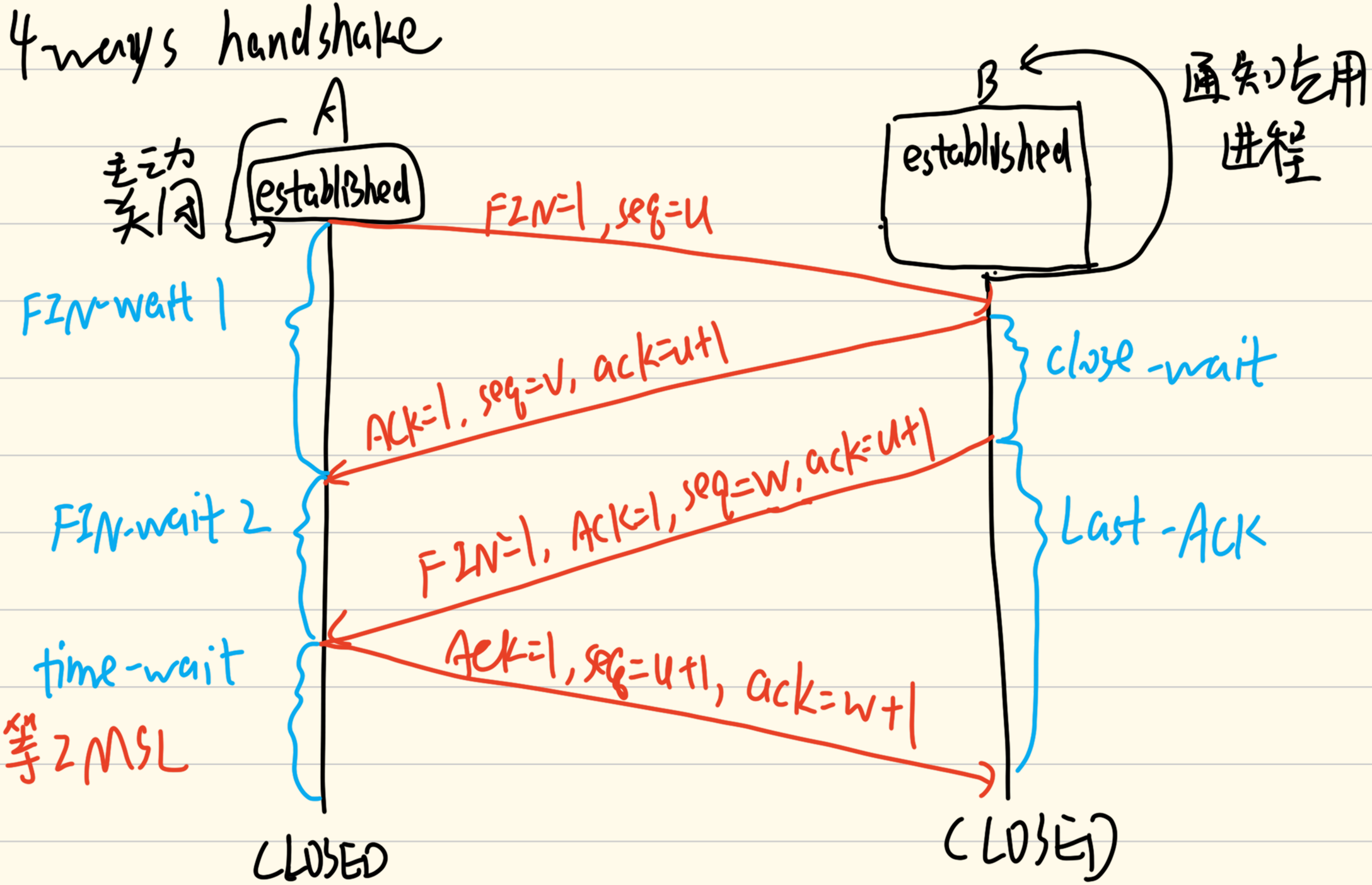
Mandatory 必需 (20B)

urg	check urgent ptr field
ack	check ack num. field
psh	push
rst	reset
syn	synchronise seq num.
Fin	no more data

3 ways handshake



4-ways handshake



socket: identifies a unique process entity on a host

↓ IP address 标 host 、 port 标 process

实现process-to-process communication

port: { 0~2^{10} well-known: 由ICANN分派、控制
2^{10}~($2^{16}-2^{14}$) registered: 向ICANN注册 防止重复
($2^{16}-2^{14}$)~2^{16} Dynamic 临时分配

↓

Socket address: TCP/IP suite 用IP address + port number来建立连接

congestion control: receiver定的默认值 拥塞控制(动态)

effective window = Min(adversed window, congestion window)

silly window syndrome: receiver window过小，sender每次只发少量data

SWS 导致传输速率低

原因：receiver处理data慢

transmiter产生data慢

策略：receiver：延迟发送window更新通知，buffer有enough空间时再发更新通知(ACK)

transmitter用 Nagle算法 sender缓存的data满足一定条件再一次性发过去

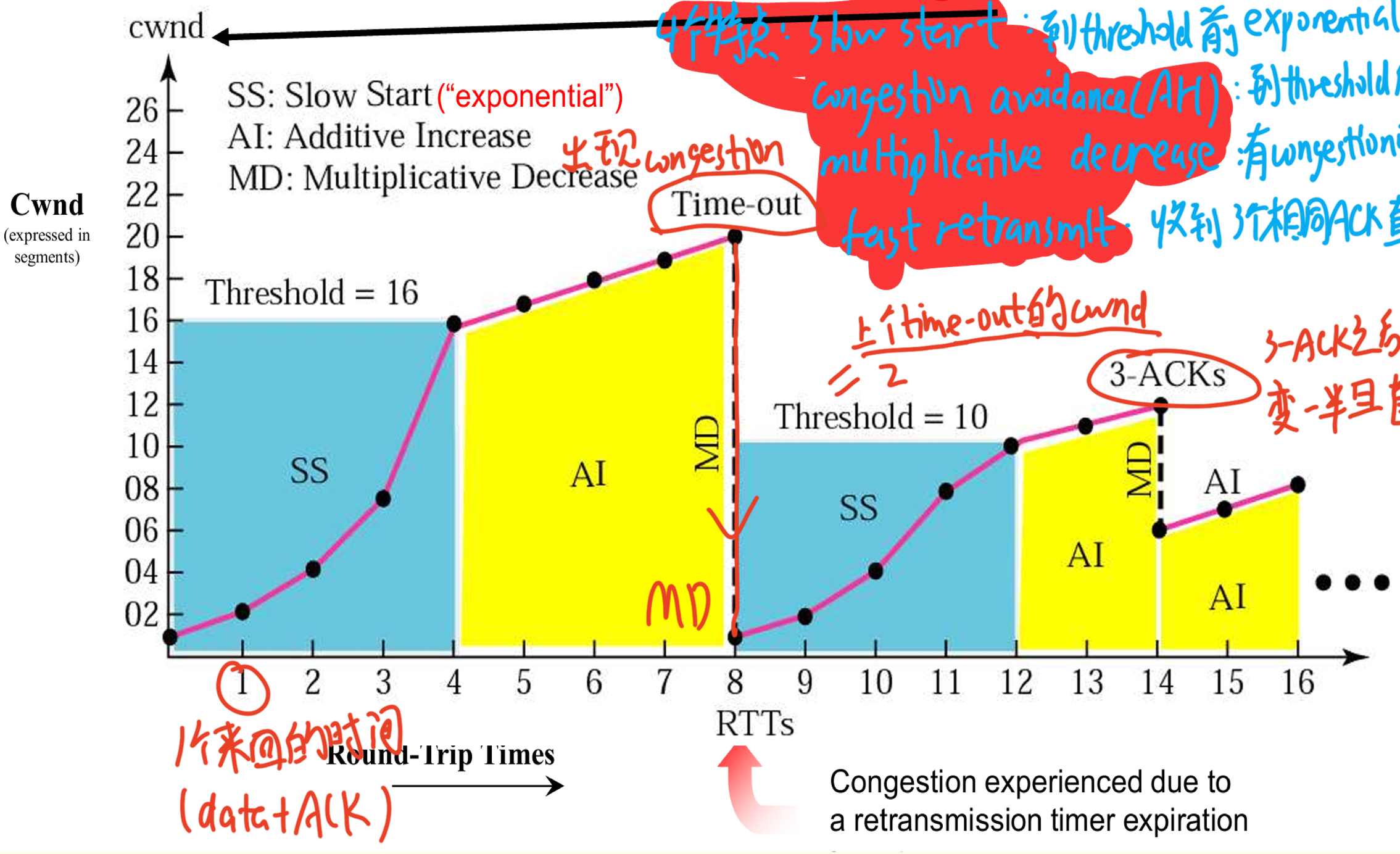
条件：① 发maximum-size segment

② 发小的segment前确保之前的所有segment都已ACK

③ receiver window at least half open

④ 有时要disable Nagle (如实时传输时)

effective window = Min [*advertised window* | *congestion window*]



4个阶段：slow start：到threshold前exponential增加

congestion avoidance(AI)：到threshold后线性加

multiplicative decrease：有congestion时cwnd=1

fast retransmit：收到3个相同ACK直接重传

出现congestion

上个time-out的cwnd

= 2

3-ACK之后cwnd变一半且直接AI

MD

1个来回的时间

(data+ACK)

retransmit timer ⟶ 用RTT (round-trip time)

Smoothed RTT

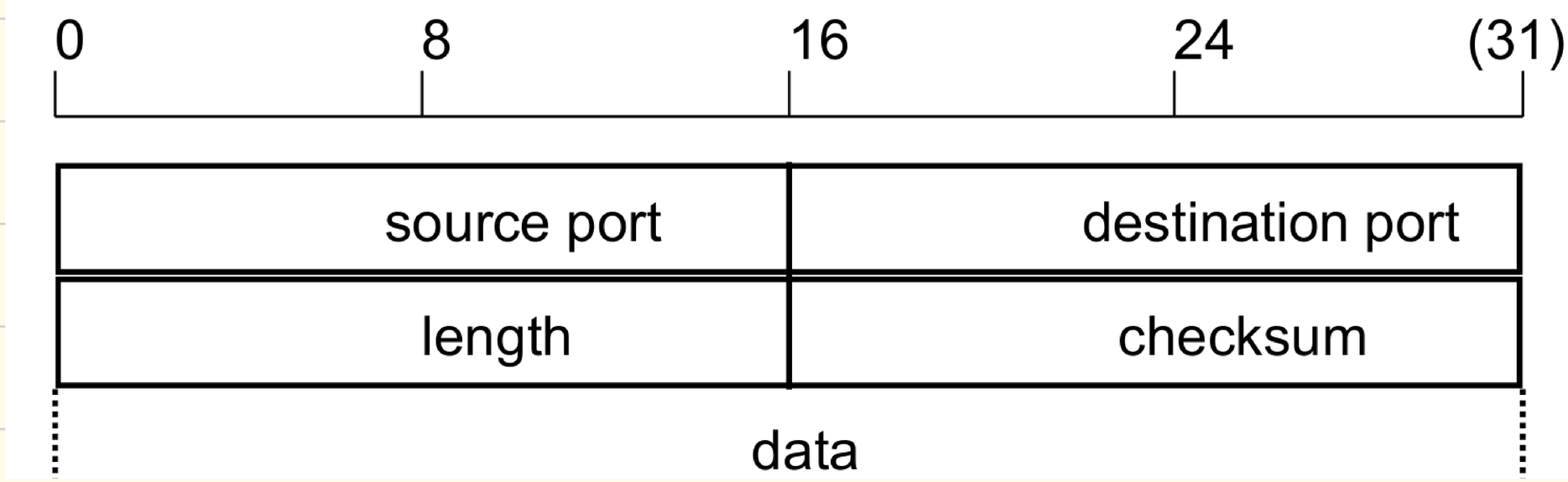
$$SRTT = (\alpha \times SRTT) + [(1-\alpha) \times RTT]$$

SRTT: 下一个
α: 超神告的系数
SRTT: 当前RTT
RTT: 上一个RTT

retransmission time-out

$$RTO = \beta \times SRTT = 2 \times SRTT$$

UDP (user datagram protocol) 用于传real-time data



① connectionless, unrealiable, unordered 的 datagram

② 无 error correction, flow control, congestion control

③ 有 error detection (checksum)

仅限于程序间不需纠错且双方需较少交互时间

优: ① no acknowledged → 快 ② low wit ③ 适合 online stream

transport layer

- UDP (user datagram protocol)
- TCP (transmission control protocol)
- ARQ (Automatic Repeat Request)

TCP:
- connection: 3-ways handshake
- finish: 4-ways handshake
- 可靠传输工作原理
  - 停止等待协议
  - 连续ARQ协议
- 可靠传输实现
  - slide window
  - time-out
- TCP flow control
- TCP congestion control
  - slow-start + AIMD
  - 拥塞避免
  - 快重传
  - 快恢复

UDP	TCP
no connection	connection-oriented
unreallable transmit 不用flow control 和congestion control	reliable transmit 用flow control congestion control
支持1对1，1对多，多对1，多对多通信	仅1对1通信
面向报文	面向字节流
header开销小(8byte)	head开销(20B~60B)
适用于实时应用	适用于要求可靠传输应用

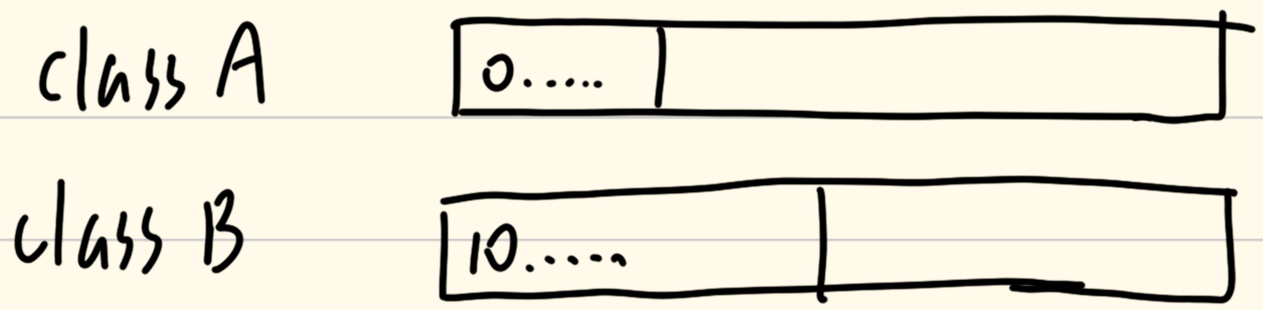
Week 2

IP protowl

available address : x.x.x.1 ~ x.x.x.254

x.x.x.0 为netid　　x.x.x.255 为broadcast

class IP address



CIDR: classless Inter-Domain Routing

↳ 用于给IP分配以、路由聚集

↳ 表示 IP address/前缀长度

124.25.3.128/22

private IP address (了解即可)

预留了：10.0.0.0～10.255.255.255

172.16.0.0～172.31.255.255

192.168.0.0～192.168.255.255

private IP 与互联网通信时要在私有网络的边缘设备(如 ADSL router)

用 NAT (network address translation)

169.254.0.0～169.254.255.255 用于自己动态用IP 分两L(APIPA)

automatic private IP addressing

127.x.x.x是loopback range用于网络测试+本地进程间通信

router: network layer device that 控制网络中 flow routing

↓ 必须有多个 physical interface 来连多个 network

↓ 用2种 protocol

routed protocol: 能将 data 封装为 IP datagram 并在 network 中传输 —— IP protocol

routing protocol: distributed dynamic 机制, used to 确定 best path flow should take

→ router 间相互协作, 确定网络中可用路由 并沿着 router 发 data

RIP、OSPF 原理是 router 间交互构建 routing table

routed protowl —— IP

作用：① 封装 TCP/UDP packet ② 提供 32bit logical address

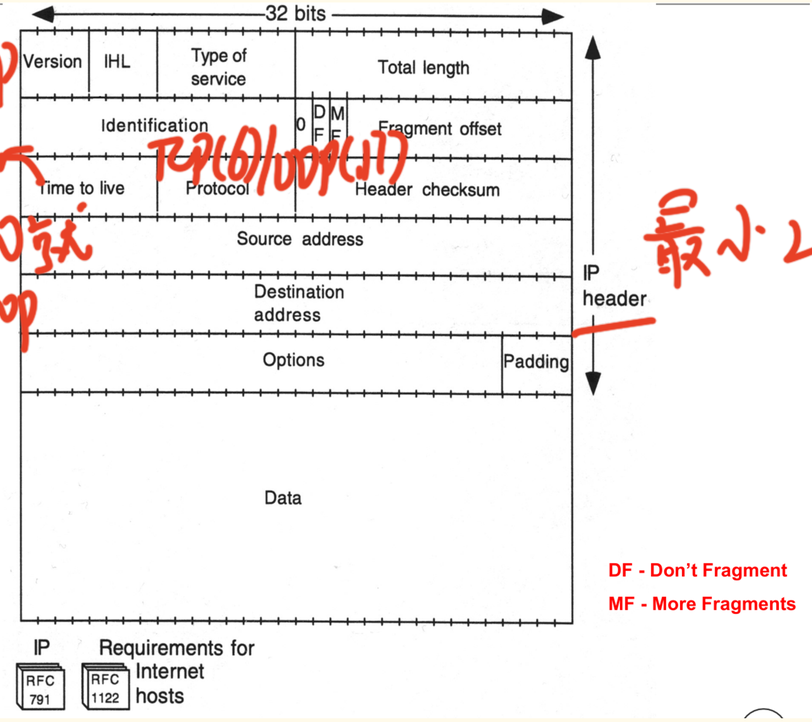
③ 作为不同network的common interface ④ routing/forwarding data的简单核心网络

优：简单 low cost

上层可构建更可靠服务

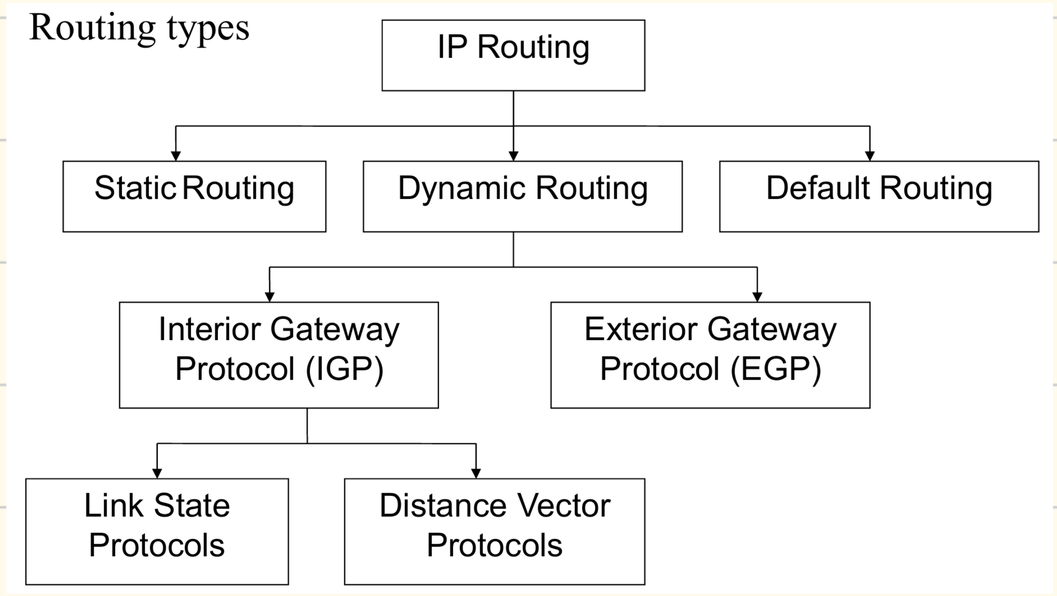
适用于许多network

The IP datagram consists of a **13-field** variable length header plus the data field itself. The maximum length of any IP datagram is **64kbytes**.



Ip 如何 fragment:

Routing



source routing: IP datagram 预先设定好一小列经过的router

static 发之前就确认了 path (不是根据routing table决定next hop)

next-hop routing: 传输中 path上每个router只根据本地routing table与datagram中

static/dynamic 信息独立选next hop, 只关心next hop, 不用了解完整path

Default route: 当无法匹配router时用默认route (没在 routing table中)

static IP routing: 管理员手动设置的在routing table中指定了path ~ Default route

每个router手动放入list of destination和对应next hop

↘优: simple、stable topology、用于规模小少变化环境

缺: not scalable、无法deal with error

Dynamic Routing: router根据network中其他router提供的信息动态计算routing table

↘为每个destination算best next hop → 算best path

↓ topology变化/error

需一段时间特变化性的有router ← convergence 优于 static

Distance vector Algorithm

node i 算到其他 node 的距离 → 发 <destination, cost> 到 neighbor → find shortest route

建 routing table for A

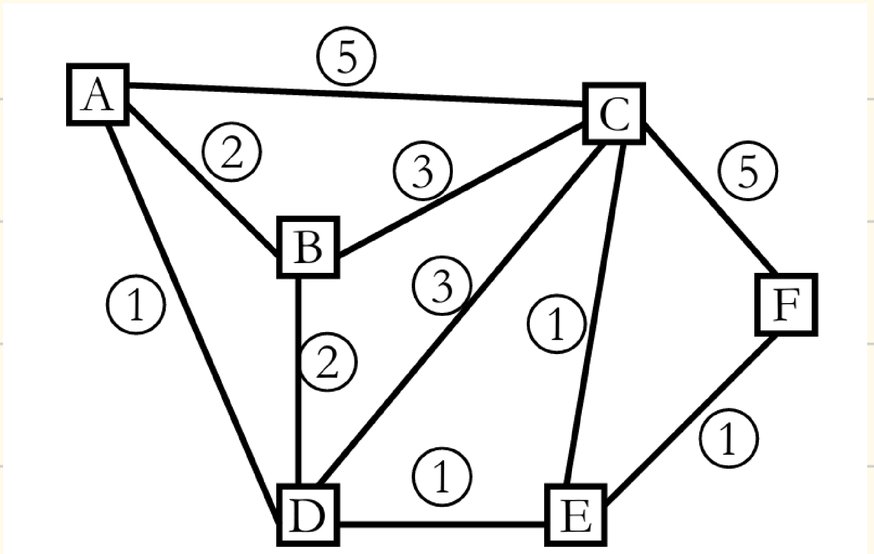


table 1：A 的 neighbor cost

node	queue length
B	2
C	5
D	1

table 2：neighbor 发给 A 的 routing table

算的值都是 shortest path

from	B	C	D
to A	2	3	1
B	0	3	2
C	3	0	2
D	2	2	0
E	3	1	1
F	4	2	2

table 3 算从 A 到其他点（E）的距离

via	1st hop distance	remaining distance	total distance
B	2	3	5
C	5	1	6
D	1	1	2

shortest

table 4 A 的 routing table

Dest.	Distance	First hop
A	0	—
B	2	B
C	3	D
D	1	D
E	2	D
F	3	D

A 到各点 shortest distance

RIP (routing Information protowl)

① distance-vector routing protowl, 用hop度量 distance between source and dest.

② interior gateway protowl (IGP)

③ max distance 15 hops (16 hops 定义 infinity 无法到达)

用UDP作为broadcast

0		31
Command(1)	Version(1)	✗ Unused - All Zeros (2)
Address Family Identifier (2)		✗ Unused - All Zeros (2)
IP Address (4)		
Unused - All Zeros (4)*		
Unused - All Zeros (4)*		
Metric (4)		

*scope for larger address fields (non-IP)

Command: 1 = Request, 2 = Response

Version: 1 for RIPv1

Address Family: 2 = IP

request → IP = destination

response → IP = source

request → metric = 0

response → metric = distance 该router → destination

可能有多条，几个link有几条

max size = 512B

2种router: active gateway 'ɜɜs broadcast 1次

passive host : listen and update, 不broadcast

① 先 broadcast 给所有 RIP-enabled interface 发 request

② 周期性 broadcast 发 response 给 neighbor (30s)

③ 收 response 后对比、更新 routing table

④ 若1条 route 180s 内未刷新，则将到该 node 的 distance 设 16，后续删除

结果：所有 router 知道任何给定 node 的 shortest path (但只知 next hop)

更新 routing table：① check for validity of the update

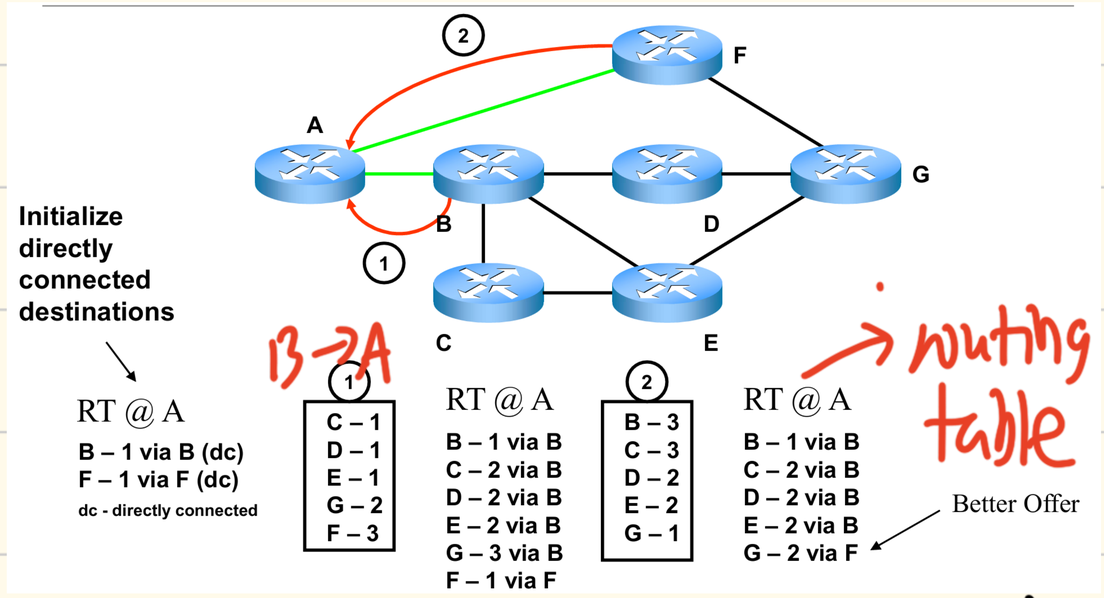
② look for corresponding destination (自己 routing table 中)

③ If destination 已存在就 update entry (if cost lower) 且 reset timeout

④ If destination 不存在 则 add 此

注：若自己 routing table 中有 lower 的，但与发来的信息的 next hop 相同，则 router 不会信，将该 next hop 设 16，并设 holddown time (防止 loop)

holddown time 结束后看 neighbor 是否还是那个更新 → 此时 router 才信 + 更新 higher cost



split horizon update: router 不会 从某接口接收的信息再从该接口发出来

poison reverse: router发现某router不可达时会在自己table中标记并广播其不可达

triggered update: route change时 router不等周期,立即向neighbor发变化

hold-down timer: router 发现该router不可达设hold-down timer, 结束前不变该router信息

缺点: ① not scalable
② infinity of 16
③ routing loop
④ robustness 差
⑤ 不支持VLSM
⑥ insecure
⑦ slow convergence

link state algorithm

Dijkstra's Algorithm 见week2 46~50页

Candidate	Cost to Root	Tree
		RA,RA,0
RA,RB,2 RA,RD,4 RA,RE,4	2 ① 4 4	RA,RA,0
RA,RD,4 RA,RE,4 RB,RC,1 ~~RB,RE,10~~	4 4 3 ② ③	RA,RA,0 RA,RB,2
RA,RD,4 RA,RE,4 RC,RF,2	4 ④ 4 5	RA,RA,0 RA,RB,2 RB,RC,1

OSPF (open shortest path first)

流程：① OSPF router 启动，通过所有接口周期发hello包

② router 收到hello后检查信息，匹配 → 添加为 neighbor

③ each router 给所有neighbor 发 LSA (link state advertisement)

④ router 从neighbor收 LSA 后存入 link state database 并发copy给其他neighbor

⑤ LSA 扩散 all area 后 所有router 建立 same Link state database

⑥ database 完成后，router用SPF算法 算 shortest path

⑦ router 从 SPF tree 中建立 routing table

node ID
SN
calculation of path

packet format 57页

message type 58页

区别

link state	distance vector
每个router收集link的有信息 构建整个network拓扑	每个router仅了解相邻router间distance、next hop 不知all network拓扑
router收集完link state后封装为LSA 广播到all network，其他router更新	router定期(30s)把routing table发给neighbor neighbor更新后再传播——环路问题
用Dijkstra算shortest path 快速响应拓扑变化、收敛快	根据neighbor发的信息，更新routing table 计算简单、收敛慢
占用较多资源(维护整个网络link state) 适用大型网络(拓扑复杂/经常变化)	占用较少资源(仅维护neighbor信息) 适用于小型网络(拓扑简单、稳定)
OSPF(open shortest path first)	RIP(routing information protocol)
事件触发更新——fast convergence	频繁、周期更新——slow convergence

subnet —— 会计算 例题见自本 2页

61~63页

step: ① 确用 network/ host 数 decimal→binary

② 在subnet mask中 reserve 1的 + 加0位 (作net id)

③ 用加之后的net id找range

exercise 见 2页

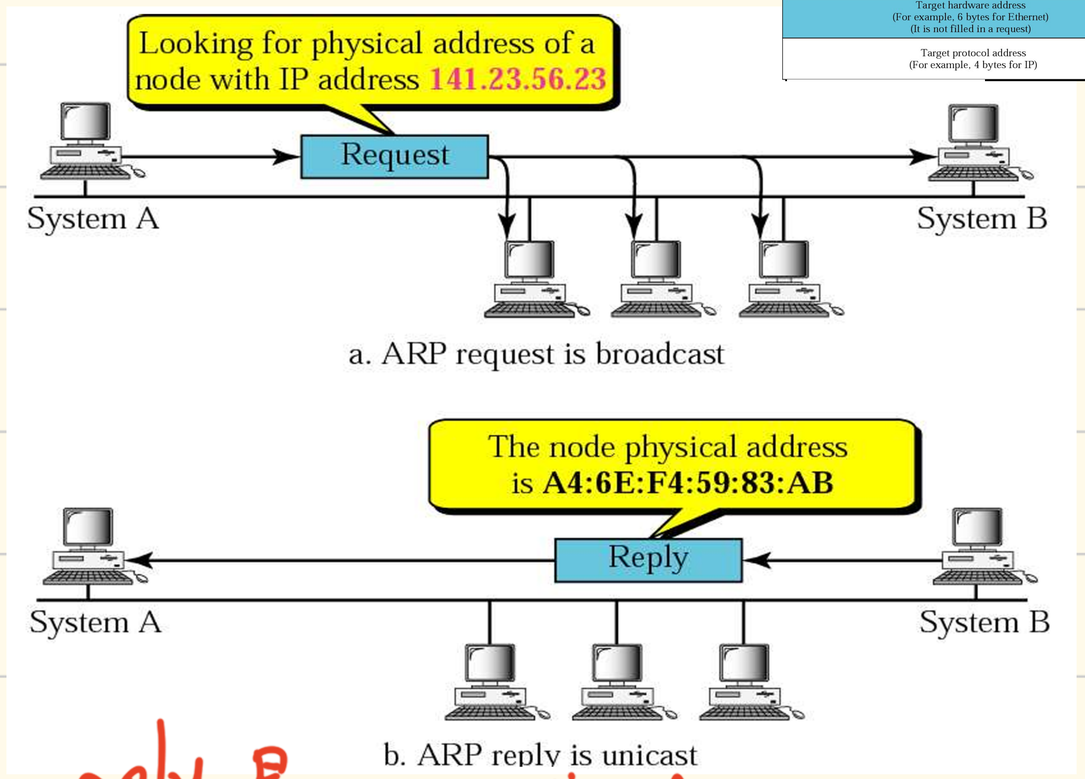
MAC — data link layer 48bit 12个hex digits

由制造商分配给network interface的 unique identifier

ARP (address resolution protocol)

IP → MAC

Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		



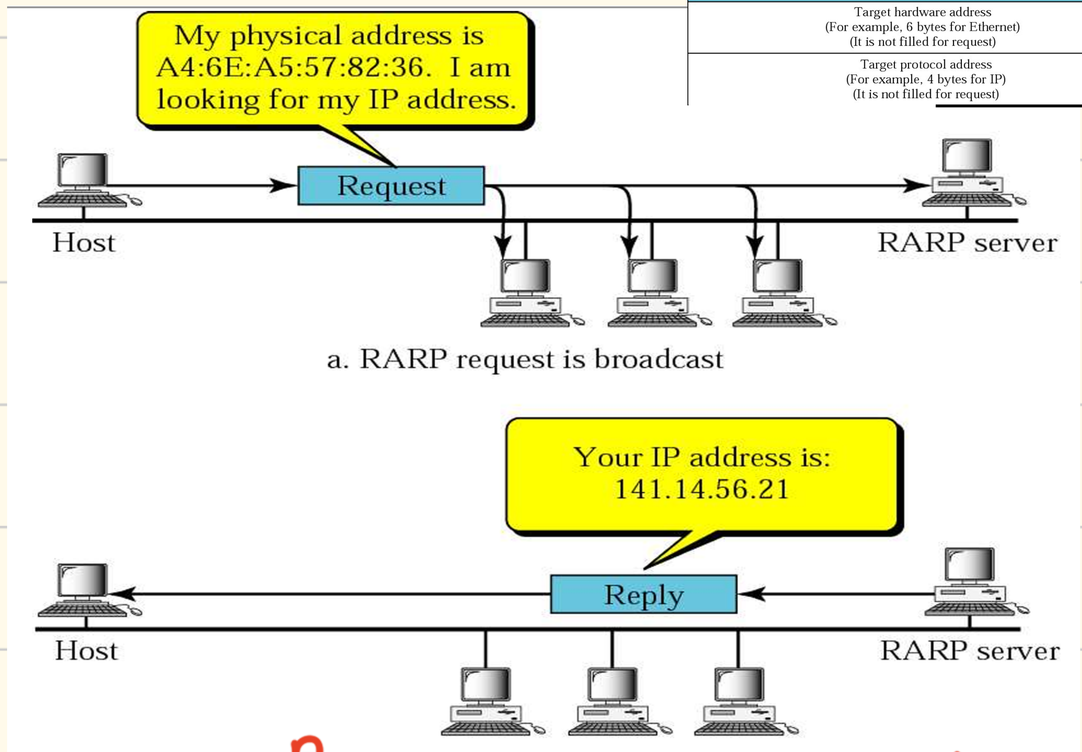
reply 是unicast发回A

request 是broadcast 找 translation server

RARP (reverse ARP)

MAC → IP

Hardware type		Protocol type
Hardware length	Protocol length	Operation Request 3, Reply 4
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP) (It is not filled for request)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled for request)		
Target protocol address (For example, 4 bytes for IP) (It is not filled for request)		



request 是broadcast 找 RARP server

reply 是 unicast 发回host

week 3

reservation based protocol collision free、公平

token ring：通过获得token 控制access
token 按顺序循环分配

优：无collision

缺 Inode failure 导致 entire channel 崩

TDMA：entire radio spectrum 分成几个 time slots，each slot 分块给 user send/receive

优 cost effective、interference free、battern life high

缺：Disconnection、同步开销high

contention based protocol 不需同步 node竞争资源

unslotted ALOHA 想传data就传 ┌ 成功 下一帧
                              └ 失效(多node同时发) 重发

如时不占进行transmit

vulnerable time = 2× frame transmission time

slotted ALOHA channel 划分成相等时段 只许 slot 开始时传输 错过等下一个

有2个以上在同一slot传 则collision

若无 collision，则下一slot 传新帧

vulnerable time = frame transmission time

efficiency = 36.8% = $\frac{1}{e}$

CSMA/CD 使用 channel前，node 先感知 shared channel

先检测后发送 → 检测channel free → 发frame

检测channel busy → wait

碰撞后 node

wait X slot-time 重传

X ∈ [0, 2^N-1] $0 \le N \le 10$

X ∈ [0, 1023] $11 \le N \le 16$

重传16次 失败后放弃

collision detection：检测出后立即停止transmit，等free时重传

repeater

① receive & retransmit 信号

② 增强 signal strength (衰减 → 初始 strength)

③ physical layer 不查 MAC、IP address

bridge — data link layer 基于 MAC 地址

将 LAN 分割成 多个 segment → 减少 network traffic + filter traffic

bridge 收到帧后检查 MAC 地址，根据转发表决定转发其他 interface / drop

若帧的 destination MAC 与接收端口不同 — 转发对应端口

将目的 MAC 与自己的 MAC 表对比 (同一段) 不转发

不同段 — 转发对应 MAC

wired Ethernet (standard IEEE 802.3)

连 wired LAN 或 WAN 的 device

优：① high speed ② better stability ③ secured connection
④ good quality transmission

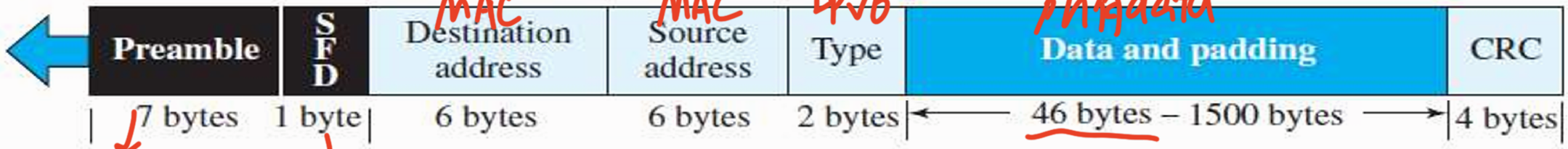
缺：① mobility 有限 ② high installation cost ③ port availability
④ long cable 导致 crosstalk

Preamble: 56 bits of alternating 1s and 0s

SFD: Start frame delimiter, flag (10101011)

IPv4 IPv6

MAC MAC 所有data



10101010.....10 10101011

↓

让 hardware 知道 beginning (但不是 frame 一部分)

wireless LAN

passive scanning: ① AP —frame→ host (host 已有记录 beacon)

active scanning (初次connect)

① host —request(广播)→ all AP

② all AP —response→ host

③ host —request(连)→ AP

④ AP —response(连)→ host

CSMA/CA: 发data前先reserve channel 防止 collision

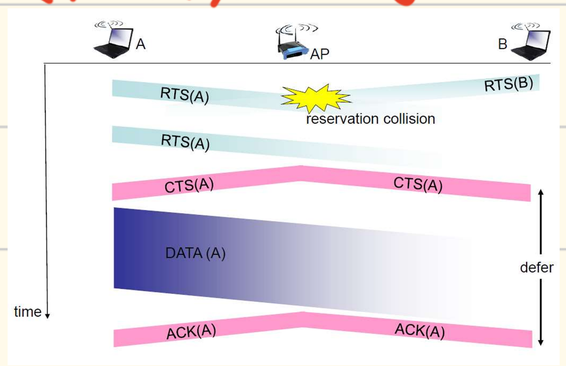
正常

① sender向BS发RTS(request-to-send)—广播,但不是发给自己的会drop(类似unicast)

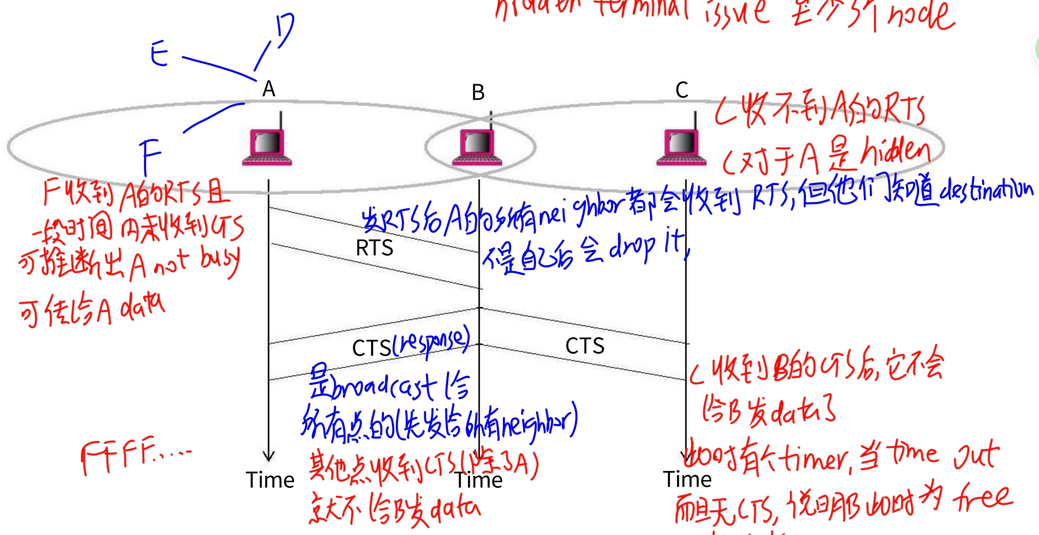
② BS收RTS后会broadcast CTS(clear-to-send)表示允许发送CTS告诉sender发送,告诉其他node停止发

③ sender收CTS后会broadcast data(但仅发CTS的先receive,其它node会drop)

④ receiver收完data发ACK

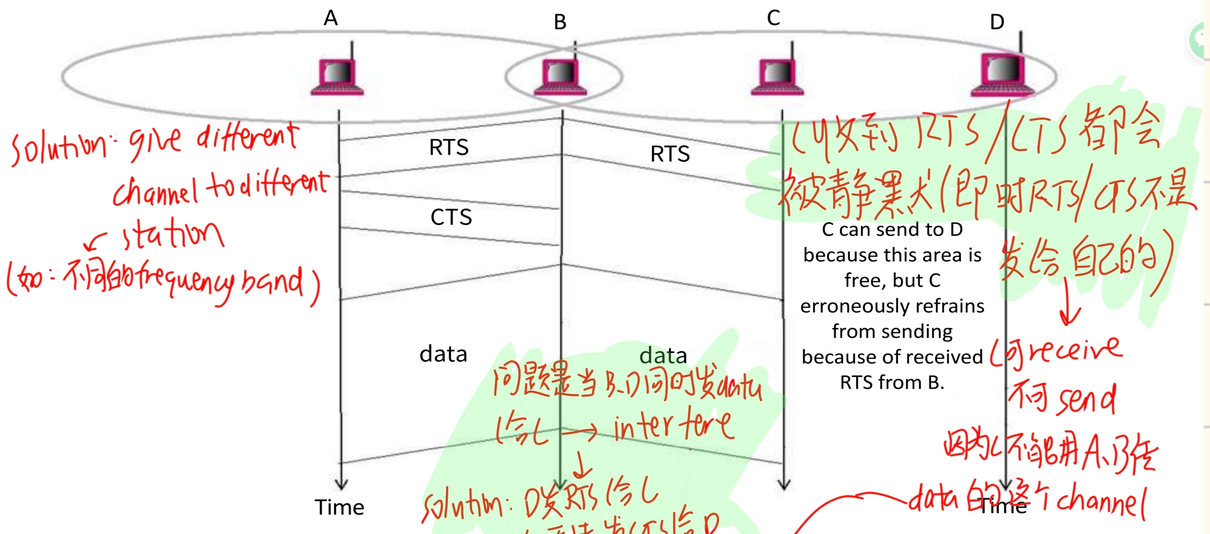


Handshake Preventing Hidden Station Problem



The CTS frame in CSMA/CA handshake can prevent collision from a hidden station

Handshaking in Exposed Station Problem



The handshaking can not help with the exposed station problem

© Queen Mary University of London

hidden terminal issue 至少3个node

C收不到A的RTS (对于A是hidden)

F收到A的RTS且一段时间内未收到CTS可推断出A not busy 可传给A data

发RTS后A的所有neighbor都会收到RTS,但他们知道destination不是自己后会drop it,

CTS(response)是broadcast传给所有点的(发送给所有neighbor)其他点收到CTS(除了A)就不传B发data

C收到B的CTS后,它不会传B发data了

此时有个timer,当time out而且无CTS,说明B此时为free可传data

FFFF....

Terminal

Solution: give different channel to different station (如:不同的frequency band)

(C收到RTS/CTS都会被静默火(即时RTS/CTS不是发给自己的)

C可receive不可send

问题是当B,C同时发data给C → interfere

Solution: D发RTS给C无法发CTS给D

因为C不能用A,B在data的这个channel

因为wireless的每个node都moveable 你不能保证C永远是hidden,不是hidden就无collision

这problem是C收不到A的RTS,对于A的hidden,C不知A→B channel是否free,可能也发data给B → collision

solution: B broadcast CTS, C收到CTS知channel busy

这problem是C不知A发的CTS,若B发data时C也发data会collision

solution: C收到RTS/CTS都会slient,即使RTS/CTS不是发给C的,此时C仅可receive,不可send

IEEE 802.11 Frame Structure: Addressing

bytes

2	2	6	6	6	2	6	0 - 2312	4
frame control	duration	address 1	address 2	address 3	seq control	address 4	payload	CRC

receiver address

Address 1: MAC address of wireless host or AP to receive this frame

Address 2: MAC address of wireless host or AP transmitting this frame

transmitter address

Address 3: MAC address of router interface to which AP is attached

router address

Address 4: used only in ad hoc mode 临时模式

IEEE 802.11ah

改进 →

spectrum 特性：good propagation、penetration

reliablity：less congested frequency，高灵敏

short beacon（信标高频低速发）

implicit ack（不用Ack）

支持上千个 associated device

MAC protocol 缺：

① collision：2个以上node在channel上同时传会撞

② overhearing：node 会receive 不相关的 packet

③ overhead：传data时有 useless data ⟶ waste energy

④ idle listening：RTS/CTS 不带有用data

Week 4 —— 会计算 看pdf

entropy $H = \sum_i p_i \log_2\left(\frac{1}{p_i}\right) = \sum_i p_i I_i$ $\quad H_{max} = \log_2 N$

source coding：用于简化编码(理论上最高效合理)

- 编 Huffman coding 画 Huffman tree

$\eta_{code} = \frac{L}{H} = \frac{\sum p_i \cdot length}{H}$ 代码有效利用资源的能力

$\eta_{source} = \frac{H}{H_{max}}$ 从source中获取有用信息的能力

Huffman coding Procedure (8-symbol)

Symbol	C	B	A	F	G	E	D	H
Probability	0.40	0.18	0.10	0.10	0.07	0.06	0.05	0.04
Codeword	1	001	011	0000	0100	0101	00010	00011
code length	1	3	3	4	4	4	5	5
$I_i = \log_2 \frac{1}{p_i}$	1.3	2.5	3.3	3.3	3.8	4.1	4.3	4.6

$H = \sum p_i I_i = 2.54$ bits/symbol

source efficiency $\eta_{source} = \frac{H}{H_{max}} = \frac{2.54}{3}$ 越低表示symbol间 dependency越高

$H_{max} = \log_2 N = \log_2 8 = 3$

average code length $L = \sum p_i \cdot \text{code length} = 2.61$ bits/symbol

code efficiency $\eta_{code} = \frac{H}{L} = \frac{2.54}{2.61}$ 越高越好 η_{code}高表示相同code length有更多信息量(效率)

$\approx \frac{I_i}{\text{code length}}$

© Queen Mary University of London

Shannon's formula: $C_{bit/s} = W \log(1+SNR) = W \log\left(1 + \frac{P}{N_0 W}\right)$ bit/s

bandwidth; signal power; noise spectral density; noise power

最后是 information 生成 cipher + 栓弦 (G、H)